Schematize: An Agentic System for Generating and Refining Information-Extraction Schemas for Legal Research

Albert Sawczyn[1†] **Jakub Binkowski**[1] **Kamil Tagowski**[1]

Łukasz Augustyniak[1] **Berenika Kaczmarek-Templin**[1] **Tomasz Kajdanowicz**[1]

[1]Wrocław University of Science and Technology
[†]albert.sawczyn@pwr.edu.pl

Abstract

Empirical legal research often relies on turning research questions into structured data extracted from large collections of rulings and judgments. Designing the extraction schema and then extracting the data remain a manual, expertise-heavy bottleneck. We present `schematize`, an open-source multi-agent system that interactively turns a researcher's problem statement into a validated extraction schema that can later be used for autonomous extraction. `Schematize` couples (i) a clarification dialogue that elicits implicit expert intent, (ii) iterative schema generation, (iii) data-grounded refinement that tests the schema against documents, and (iv) chat-based post-editing. We evaluated the system with human legal professionals, introducing our novel methodology, and `schematize` achieves top performance in most of tested configurations. While the system is designed to be domain-agnostic and applicable to any document collection, we tailor and evaluate it on legal research problems. We release `schematize` as a pip-installable Python package with full documentation.

1 Introduction

Empirical legal research often depends on structured representations of case law, rulings, and judgments: to study a question quantitatively, researchers must first extract fields that can be systematically analyzed across hundreds or thousands of documents (Hwang et al., 2022; Mali et al., 2024). Production legal-analytics platforms likewise combine large-scale document retrieval, expert-defined criteria, and structured extraction to support quantitative studies (Augustyniak et al., 2026). The quality of the outcomes depends strongly on how researchers formulate the research problem and then design the extraction schema that specifies *what* to pull from each document. This upstream formulation is itself difficult: an initial request often leaves scope, entities, exclusions, comparisons, and downstream analytical goals implicit, even when the researcher has a clear study in mind. The process demands both legal expertise and data-modeling skills, from defining atomic fields to judging which values an LLM can reliably extract, while being slow, iterative, and prone to bias. For instance, a schema that omits a relevant field silently caps what the downstream study can ever discover.

Modern information-extraction systems share a common interface: they take a user-supplied *schema* that declares what to pull from each document and ask a model to fill it in (Zaratiana et al., 2025; Shrimal et al., 2025; Goel, 2025), and the same assumption underlies legal IE pipelines (C R et al., 2024; Mali et al., 2024). A growing line of work instead tries to *derive* schemas from a research question and a corpus (Levy et al., 2026; Sadruddin et al., 2025; Padmakumar et al., 2025), but it typically validates against a reference schema and offers little support for grounding the schema in the documents it will be applied to. The upstream step – turning a research question into a good schema and checking it against real data – is thus left to the analyst, with no agreed way to tell a good schema from a bad one (Section 2).

`Schematize` closes this gap with a human-in-the-loop, multi-agent pipeline (Figure 1). A researcher states a problem in natural language; a *problem-definition-helper* agent asks clarifying questions, and once the expert answers, a *definer* agent turns the request and responses into a formal problem definition. The system then generates search queries, drafts an initial schema, and improves it through two refinement loops: a criteria-based refinement loop in which a *critic* agent scores the schema against quality criteria, and a data-grounded refinement loop that retrieves real documents with the generated queries and tests whether the schema can actually be populated

from them. Finally, a summary explains how the schema was derived, and an interactive chat lets the expert request further edits. The result is an extraction schema that is grounded in both expert intent and real documents before any large-scale extraction begins. Notably, we provide a novel methodology for the evaluation of schema quality by asking human experts to write exhaustive question sets for each research problem, which is specifically designed to limit the impact of biases in the evaluation process (Section 4), and then scoring the schema for coverage by an LLM-as-judge (Zheng et al., 2023).

- **System & library:** an open-source, pip-installable library for research-problem-to-schema generation that is model-agnostic – any agent runs on either open- or closed-weight models – with pluggable data connectors and a schema-based extractor.

- **Evaluation methodology & data:** an expert-question coverage protocol – experts write exhaustive question sets for each research problem (∼15–30), and schemas are scored for coverage by an LLM-as-judge – together with expert-curated legal research problems and their evaluation-questions sets.

- **Empirical study:** a comprehensive evaluation on legal research problems, comparing small and large LLMs, single- and multi-agent configurations, system ablations, and cost–quality trade-offs.

Availability. We publish `schematize` as a pip-installable package[1] with documentation[2] and source code[3] available, all under the MIT license. The dataset containing all judgments used in the study is available in the Hugging Face repository.[4]

2 Related Work

Information extraction (IE) turns unstructured documents into structured records (Jurafsky and Martin, 2026). As large language models have made extraction increasingly reliable (Bai et al., 2024), the field has settled on a common interface: rather than hand-building an extractor for each task, modern systems take a user-supplied *schema* that declares what to pull from each document and ask the model to fill it in (Zaratiana et al., 2025).

Schema-driven information extraction treats this schema as the contract between user and model. For instance, GLiNER2 unifies entity recognition, classification, and structured extraction behind a schema-based API (Zaratiana et al., 2025); PARSE optimizes existing JSON schemas for reliable LLM extraction (Shrimal et al., 2025); LangExtract grounds each extracted field in its source span (Goel, 2025). The same assumption carries into legal IE, where systems extract entities and relations against task-specific schemas to support quantitative case-law analysis (C R et al., 2024; Hwang et al., 2022; Mali et al., 2024).

Deriving schemas, rather than assuming them, is the concern of the work closest to ours. ScheMatiQ (Levy et al., 2026) turns a research question and a corpus into a query-driven schema and creates a grounded database through interactive discovery; SchemaMiner builds schemas from scientific documents with human-in-the-loop refinement (Sadruddin et al., 2025); intent-conditioned methods generate and refine literature-review table schemas (Padmakumar et al., 2025); SciDaSynth assembles structured tables from multi-modal scientific sources (Wang et al., 2025); and LOGOS induces hierarchical schemas for qualitative grounded-theory analysis (Pi et al., 2025).

Our work differs from prior work along three axes. First, rather than assuming a corpus, `schematize` generates retrieval queries from the problem statement and pulls documents through pluggable connectors, so schema design is decoupled from having a pre-assembled collection. Second, we separate a document-blind, criteria-based critique – guarding against overfitting to any particular sample – from a data-grounded refinement loop, whereas the systems above either skip document grounding or interleave it directly with generation. Third, and most importantly, we evaluate schemas against expert-authored *question sets* rather than treating reference schemas as gold: closely related work often evaluates by reconstructing manual or table-derived schemas, even though such references can be ambiguous, shaped by feasibility constraints, or contain artifacts (Levy et al., 2026; Padmakumar et al., 2025).

[1] https://pypi.org/project/schematize/
[2] https://pwr-ai.github.io/schematize/
[3] https://github.com/pwr-ai/schematize
[4] https://huggingface.co/datasets/JuDDGES/pl-court-raw (CC BY 4.0 license)

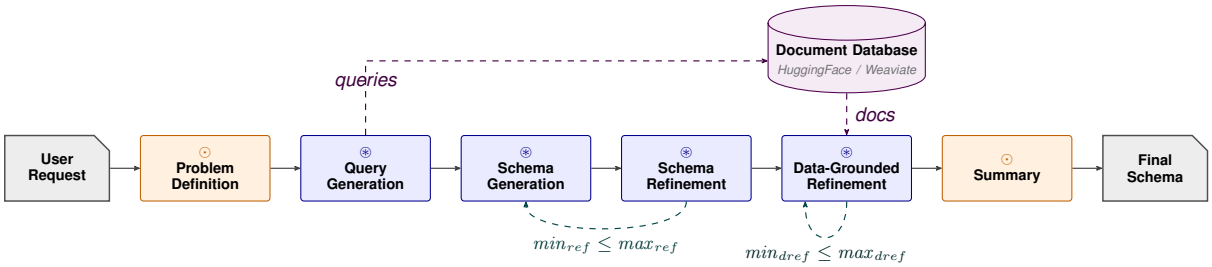


Figure 1: Overview of `schematize`'s agentic pipeline. A user request is clarified, formalized into retrieval queries and an initial schema, refined via criteria-based (CBR, up to max_{ref} iterations) and data-grounded (DGR, up to max_{dref} iterations) loops, and finalized through a summary and interactive chat. Blue (⊛) marks automated agent steps, orange (⊙) marks human-in-the-loop steps, violet marks the document store, and teal marks refinement loops.

A question-coverage protocol is repeatable across system versions without re-annotation. Moreover, a good, robust schema for a given research problem can be constructed in multiple valid ways – field names, granularity, and typing choices vary even among experts solving the same problem – so schema design has no single ground-truth target; this is precisely why we evaluate against expert-authored questions rather than a single reference schema.

Empirical legal research is our main focus, where automation must augment rather than replace expert judgment (Ramesh et al., 2025). Prior legal IE work delivers strong extractors but, as above, leaves schema design to the analyst (Mali et al., 2024; Hwang et al., 2022). `Schematize` fills this gap with a human-in-the-loop pipeline that helps legal researchers express, ground obtained schema before any extraction begins.

3 The `schematize` System

`Schematize` is organized as a pipeline of LLM agents (Figure 1). Each agent has a single responsibility and communicates through structured messages. Users can configure the pipeline by choosing LLMs (either closed-weight API models or open-weight models), data connectors, and other parameters described in the library documentation. We describe each pipeline stage below. Notably, we tailored the prompts with the help of legal experts to be more specific to the legal domain.

Problem Definition First, the legal researcher sketches a problem of interest (e.g., "study personal-rights violations and assess their severity"). A *problem-definition-helper* (PDH) agent reads this request and asks a short batch of clarifying questions – about scope, jurisdiction, the granularity of the target variables, and intended downstream analysis – surfacing implicit assumptions an expert could leave unstated. The expert answers, and a *problem-definer* agent compiles the request and prepares a formal problem definition, including a problem statement, the legal domain, the scope of judgments of interest, the legal concepts involved, and a set of research questions.

Query Generation From the formal problem definition, a *query-generator* agent produces search query to find the documents the schema will eventually be applied to. These queries feed the document retriever in the data-grounded loop (Section 3).

Schema Generation and Criteria-Based Refinement A *generator* agent first drafts an extraction schema from the problem definition; Listing 1 shows an excerpt. The schema then enters a criteria-based refinement (CBR) loop: an *assessment* agent scores it for coverage of the problem definition, field clarity and non-redundancy, appropriate typing and granularity, and extractability, and a *refiner* agent applies the recommended changes. To avoid overfitting the schema to the retrieved documents, the *generator* and *refiner* agents operate without document access and rely only on the problem-definition instructions. The loop repeats until it runs at least min_{ref} rounds and stops once the assessment reports no further changes are needed or a cap of max_{ref} rounds is reached.

Data-Grounded Assessment and Refinement The second loop grounds the schema in the context of real documents. Using the generated queries, a document retriever fetches a set of matching legal documents through a configured backend, such as Hugging Face datasets or a Weaviate vector store.

This data-grounded refinement (DGR) loop starts when a *data-assessment* agent examines each retrieved document to assess the schema from a practical perspective, and a *merger* agent consolidates these per-document assessments into a single set of revisions. Finally, a *data-refiner* agent updates the schema accordingly. Like the first loop, it runs between min_{dref} and max_{dref} rounds, with the *data-assessment* agent examining a configurable number of retrieved documents per round.

Summarization and Interactive Chat After the refinement loops, a *summarizer* agent produces a short report explaining how the schema was derived – which fields were added or changed in each loop and why – so the expert can audit the process rather than trust an opaque output. The session then enters an open-ended chat: the expert can ask for further changes in natural language, and a chat agent edits the schema in place, keeping the human in control of the final artifact.

3.1 Library Design

`Schematize` ships as an installable Python package so the pipeline can be run programmatically or embedded in other tools. LLM access goes through LiteLLM (Dholakia and Jaffer, 2023), and orchestrated through the langgraph framework (LangChain Inc., 2024), making the library modular and model-agnostic: agents can be backed by either open-weight models (e.g., self-hosted Llama or Qwen run locally or behind a vLLM (Kwon et al., 2023)/Ollama (Ollama, 2023) endpoint) or closed-weight API models (e.g., GPT, Claude, or Gemini). This lets users trade costs, latency, and data-privacy requirements against capability. Data access is abstracted through an abstract retriever base class, which ensures that the DGR loop receives documents in a uniform format regardless of their source. The library includes connectors for Hugging Face datasets and Weaviate, and supports other document sources through extensions provided by users.

4 Empirical Evaluation

We introduce a custom methodology for assessing the quality of generated schemas while limiting potential annotation biases. In particular, we evaluate the system on three Polish legal research problems in collaboration with seven legal experts.

4.1 Methodology

In prior studies, experts were asked to assess schemas generated by the evaluated systems. However, such holistic and intuitive evaluation can introduce confirmation and hindsight biases among annotators (Mahdavi and Rahimian, 2017). Instead, we propose a protocol wherein domain experts cannot see the generated schema but independently write the atomic questions that the extracted data should be able to answer (see Figure 5). The experts wrote questions until they judged the topic to be exhausted; we suggested approximately 15–30 questions per problem. By asking experts to formulate questions upfront, we shift their cognitive processing from post-hoc schema evaluation to direct reasoning about the problem, mitigating potential biases. For each research problem, experts wrote their questions from a single fixed PDH exchange: a user request, PDH clarifying questions from gpt-5.2, and the user's answers (Figure 5). In our study, we recruited 7 experts from the District Chamber of Legal Advisers in Wrocław and asked them to annotate three problems. We randomly assigned 3 experts to each problem. To ensure that experts understood the annotation task and were aligned on the protocol, we prepared detailed instructions (see Appendix B) and conducted a workshop session to explain the details and answer questions.

4.2 Model configuration and baselines

We compare `schematize` with two baselines: a *vanilla LLM*, which generates a schema in a single call, and the multi-agent *ScheMatiQ* system (Levy et al., 2026). Because `schematize` includes the PDH stage, it receives information elicited from the user beyond the initial problem sketch. To isolate the contribution of the pipeline rather than this additional input, we evaluate each baseline both without and with the same fixed PDH exchange (Figure 5). This yields four baseline configurations: *vanilla LLM*, *vanilla LLM + PDH*, *ScheMatiQ*, and *ScheMatiQ + PDH*; the PDH-augmented configurations generate schemas from the clarified problem. Our study uses both closed-weight models (gpt-5.4, gpt-5.4-mini, gpt-5.4-nano, claude-sonnet-4.6) and open-weight models (qwen3.6-35b-a3b, gemma-4-e4b-it, llama-4-scout-17b) of varying sizes[5]. In Section 4.4, we report coverage

[5]Licenses (all permit research use): gpt-5.4/-mini/-nano: OpenAI Services Agreement; claude-sonnet-4.6: Anthropic

results for the best hyperparameter configuration found, and provide a broader hyperparameter sensitivity study in Section 5.1 along with component ablations in Section 5.2.

4.3 Metrics

We report **coverage**: the fraction of expert questions that the generated schema can answer, judged per question by an LLM-as-judge (gpt-5.4-mini) that checks whether the schema contains the fields needed to resolve each question. LLM-as-a-judge evaluation is widely used and has demonstrated substantial alignment with human judgments in prior evaluation settings (Zheng et al., 2023; Thakur et al., 2025; Janiak et al., 2025). Thus, coverage provides a useful automatic proxy for schema quality.

4.4 Results

Table 1 reports coverage across model sizes, with a complementary visualization in Figure 2, comparing `schematize` with vanilla LLM and ScheMatiQ baselines. `Schematize` consistently outperforms both the vanilla LLM and ScheMatiQ baselines, as well as their PDH-augmented variants, across all tested backbones: it obtains the best or second-best result in 6 of the 7 tested configurations, including the best result in 4 configurations. This pattern holds for both open- and closed-weight models, suggesting that the proposed method remains effective in settings where privacy or deployment constraints favor open-weight models. Although larger models generally improve coverage, `schematize` surpasses the average inter-expert coverage (Table 2) on 4 of the 7 tested backbones. In other words, `schematize` anticipates more of an expert's questions than the other experts do themselves, on average. On qwen3.6-35b-a3b and llama-4-scout-17b, ScheMatiQ + PDH edges out `schematize` by only $\sim$1pp, and that margin comes from PDH – our own contribution – not from ScheMatiQ itself.

In addition, Figure 3 shows that subsequent schema refinement iterations improve the coverage of the generated schema. However, these gains plateau after several iterations, suggesting that comparable coverage can be achieved without many expensive refinement steps. As Figure 3 also shows, small models like gpt-5.4-nano struggle to grasp that their step is part of a larger pipeline: coverage drops sharply once the schema enters the data-grounded refinement (DGR) loop. On our inspection, the model appears to overfit to whichever document it is currently looking at, dropping previously established fields that are not reflected in that document instead of merging the new evidence with the existing schema. Further, we present detailed results in Appendix C, inter-expert coverage in Table 2, and inference costs in Table 5.

Commercial Terms; qwen3.6-35b-a3b: Apache 2.0; gemma-4-e4b-it: Apache 2.0; llama-4-scout-17b: Llama 4 Community License.

5 Analysis

5.1 Hyperparameter Sensitivity

Our system is parametrized by the minimum and maximum number of CBR rounds, min_{ref} and max_{ref}, the minimum and maximum number of DGR rounds, min_{dref} and max_{dref}, and the number of documents the *data-assessment* agent examines per DGR round. To select the best hyperparameters, we run a grid search with gpt-5.4-nano and choose the configuration with the highest coverage.

We report coverage for the tested configurations in Figure 4. Coverage remains stable across several hyperparameter settings, suggesting that `schematize` is robust to these choices. This stability also indicates that users can often reduce costs by skipping hyperparameter search, especially when evaluation data is unavailable and the goal is schema generation. The best configuration, used for the main results in Table 1, sets $min_{ref} = 2$, $max_{ref} = 3$, $min_{dref} = 2$, and $max_{dref} = 4$, examining a single document per DGR round.

5.2 Ablation Study

We assess the contribution of each pipeline stage by selectively ablating the problem-definition-helper (PDH), the criteria-based refinement (CBR) loop, and the data-grounded refinement (DGR) loop, individually and in combination; removing all three collapses `schematize` into the single-LLM baseline (the vanilla LLM setting in Table 1). Table 4 shows that removing CBR or DGR components has only a modest effect, well within noise, whereas removing PDH substantially decreases the performance. Likewise, removing two components together consistently leads to degradation, which indicates that they are complementary and `schematize` peaks when PDH, CBR, and

Backbone	vanilla LLM	vanilla LLM + PDH	ScheMatiQ	ScheMatiQ + PDH	schematize
gpt-5.4	36.3% ± 11.7pp	64.3% ± 19.7pp	43.8% ± 15.6pp	64.3% ± 17.6pp	**79.4% ± 12.6pp**
gpt-5.4-mini	31.7% ± 7.2pp	55.5% ± 14.9pp	23.9% ± 7.9pp	57.6% ± 7.6pp	**74.8% ± 7.2pp**
gpt-5.4-nano	22.2% ± 18.0pp	**59.8% ± 10.3pp**	24.0% ± 13.8pp	42.5% ± 20.4pp	34.7% ± 35.0pp
claude-sonnet-4.6	37.1% ± 10.8pp	60.5% ± 12.9pp	49.6% ± 29.2pp	67.1% ± 14.8pp	**70.0% ± 18.5pp**
qwen3.6-35b-a3b	28.5% ± 8.7pp	51.6% ± 2.5pp	40.4% ± 15.6pp	**67.2% ± 15.3pp**	66.5% ± 16.3pp
gemma-4-e4b-it	3.8% ± 4.0pp	38.2% ± 14.8pp	34.5% ± 18.2pp	46.5% ± 4.8pp	**53.4% ± 3.7pp**
llama-4-scout-17b	27.7% ± 20.0pp	42.0% ± 8.8pp	32.7% ± 19.3pp	**45.3% ± 10.1pp**	45.1% ± 7.9pp

Table 1: Expert-question coverage (mean ± std, in pp) across backbones and configurations. "+ PDH" prepends the Problem-Definition-Helper stage to a baseline. Bold marks the best, underline the second-best, coverage per backbone.

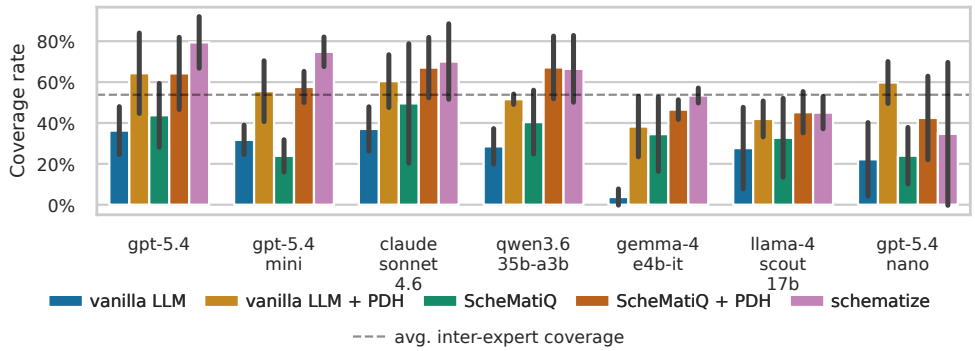


Figure 2: Comparison of the `schematize` to all tested baselines. The dashed line marks the average inter-expert coverage (Table 2).

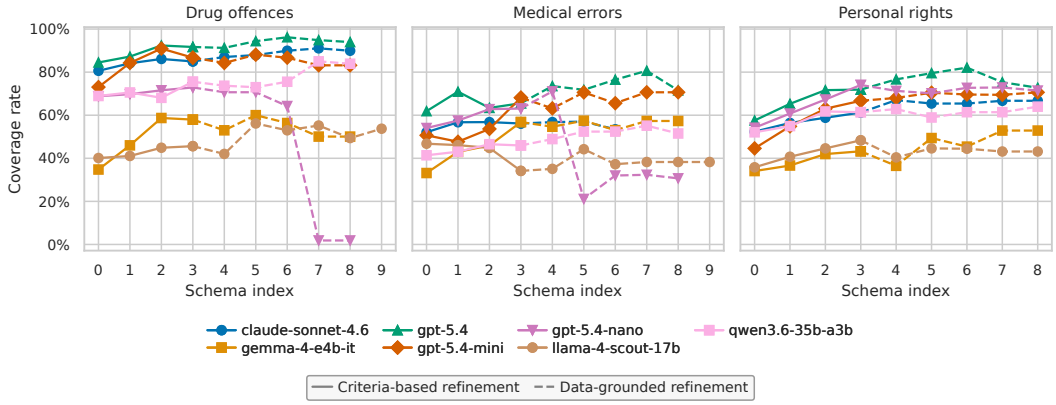


Figure 3: Coverage change over subsequent iterations of schema refinements for three considered problems.

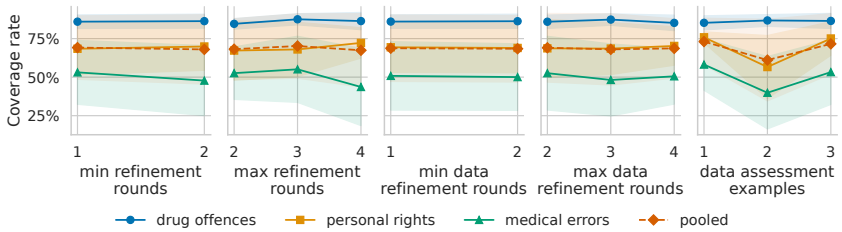


Figure 4: Results of the hyperparameter sensitivity study. We measure coverage of the tested configurations for three legal research problems.

DGR operate together. The full system's larger variance reflects differences in problem difficulty rather than component instability. Ablation runs are separate from the main results, so the "Full (`schematize`)" row may differ slightly due to LLM non-determinism.

6 Conclusion

We contributed `schematize`, a human-in-the-loop multi-agent system that turns a research problem into a document-grounded extraction schema through clarification, criteria-based critique, and data-grounded refinement against retrieved documents. We rigorously tested the system by introducing a novel methodology to evaluate the schema quality with help of human legal professionals. Empirical legal scholars and legaltech builders can use `schematize` to generate extraction schemas from research problems without hand-crafting them first. The code and pip-installable package are publicly available.

7 Limitations

Our evaluation is based on a single jurisdiction and language, and performance may vary for others. We focused on Polish case law because we were able to recruit legal experts with relevant expertise in this jurisdiction.

Coverage is scored by an LLM-as-judge, so the final estimate depends on the judge model and prompt. To limit biases that can arise when assesing a schema post-generation, experts authored question sets defining the target information needs prior to generation. However, we did not compare the LLM judge's individual coverage decisions with human annotations.

Additionally, coverage measures schema only recall quality: whether the generated fields cover expert information needs. While extra fields may hinder extraction of the fields of interest, they may also provide broader perspective and have only negligible effects on extraction quality. Also, our evaluation measures schema coverage rather than end-to-end extraction accuracy.

8 Ethics Statement

`Schematize` targets the legal domain, a high-stakes setting where automation must augment rather than replace expert judgment (Ramesh et al., 2025). It is a research tool for designing extraction schemas and producing structured data for analysis; it does not provide legal advice, and its outputs should not be relied on for legal decisions. Because automatically generated schemas can look authoritative, we keep the human in the loop at both problem definition and final editing, and the summarizer makes the derivation auditable to discourage over-reliance.

The study with human experts was carried out in collaboration with the District Chamber of Legal Advisers in Wrocław, Poland, under a formal agreement between the Chamber and Wrocław University of Science and Technology. The experts participated voluntarily and without compensation, with the exception of the study coordinator, who is an employee funded by the National Science Centre grant.

Acknowledgments

Annotators We gratefully thank the legal experts from the District Chamber of Legal Advisers in Wrocław, Poland, who contributed the annotations: Monika Bocheńska, Dominika Fikus, Dorota Jarzębowska, Berenika Kaczmarek-Templin, Małgorzata Kozłowska, Maciej Kusyk, and Mateusz Lato. Their domain expertise was essential for constructing the evaluation protocol and expert question sets.

Funding This work was co-funded by the National Science Centre, Poland, under CHIST-ERA Open & Re-usable Research Data & Software (grant no. 2022/04/Y/ST6/00183). We gratefully acknowledge the Wroclaw Center for Networking and Supercomputing for providing computing facilities and support.

References

Łukasz Augustyniak, Kamil Tagowski, Adrian Szymczak, Jakub Binkowski, Albert Sawczyn, Michał Skibiński, Denis Janiak, Mateusz Bystroński, Grzegorz Piotrowski, Michał Bernaczyk, Krzysztof Kamiński, and Tomasz Jan Kajdanowicz. 2026. Bridging AI and law: A scalable multi-agent platform for quantitative legal analytics across millions of documents. In *Bridge between Artificial Intelligence and Law (AILaw)*, pages 207–214.

Fan Bai, Junmo Kang, Gabriel Stanovsky, Dayne Freitag, Mark Dredze, and Alan Ritter. 2024. Schema-driven information extraction from heterogeneous tables. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10252–10273. Association for Computational Linguistics.

Chaitra C R, Sankalp Kulkarni, Sai Rama Akash Varma Sagi, Shashank Pandey, Rohit Yalavarthy, Dipanjan Chakraborty, and Prajna Upadhyay. 2024. LeGen: Complex information extraction from legal sentences using generative models. In *Proceedings of the Natural Legal Language Processing Workshop 2024*, pages 1–17, Miami, FL, USA. Association for Computational Linguistics.

Krish Dholakia and Ishaan Jaffer. 2023. Litellm: A unified interface to call 100+ llm apis. MIT License.

Akshay Goel. 2025. LangExtract: LLM-powered structured information extraction from text with source grounding. https://github.com/google/langextract. Open-source software, Apache-2.0 license. DOI: 10.5281/zenodo.17015089.

Wonseok Hwang, Saehee Eom, Hanuhl Lee, Hai Jin Park, and Minjoon Seo. 2022. Data-efficient end-to-end information extraction for statistical legal analysis. In *Proceedings of the Natural Legal Language Processing Workshop 2022*, pages 143–152, Abu Dhabi, United Arab Emirates (Hybrid). Association for Computational Linguistics.

Denis Janiak, Jakub Binkowski, Albert Sawczyn, Bogdan Gabrys, Ravid Shwartz-Ziv, and Tomasz Jan Kajdanowicz. 2025. The illusion of progress: Re-evaluating hallucination detection in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 34728–34745, Suzhou, China. Association for Computational Linguistics.

Daniel Jurafsky and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd edition. Online manuscript released January 6, 2026.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626. Association for Computing Machinery.

LangChain Inc. 2024. LangGraph: Low-level orchestration framework for building stateful, multi-actor applications with llms. https://github.com/langchain-ai/langgraph. Accessed: 2026-07-07.

Shahar Levy, Eliya Habba, Reshef Mintz, Barak Raveh, Renana Keydar, and Gabriel Stanovsky. 2026. ScheMatiQ: From research question to structured data through interactive schema discovery. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 220–230, San Diego, California, United States. Association for Computational Linguistics.

Shaudi Mahdavi and M. Amin Rahimian. 2017. Hindsight bias impedes learning. In *Proceedings of the NIPS 2016 Workshop on Imperfect Decision Makers*, volume 58 of *Proceedings of Machine Learning Research*, pages 111–127. PMLR.

Drish Mali, Rubash Mali, and Claire Barale. 2024. Information extraction for planning court cases. In *Proceedings of the Natural Legal Language Processing Workshop 2024*, pages 97–114, Miami, FL, USA. Association for Computational Linguistics.

Ollama. 2023. Ollama: Get up and running with large language models locally. https://github.com/ollama/ollama.

Vishakh Padmakumar, Joseph Chee Chang, Kyle Lo, Doug Downey, and Aakanksha Naik. 2025. Intent-aware schema generation and refinement for literature review tables. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 23450–23472, Suzhou, China. Association for Computational Linguistics.

Xinyu Pi, Qisen Yang, and Chuong Nguyen. 2025. LOGOS: LLM-driven end-to-end grounded theory development and schema induction for qualitative research. *Preprint*, arXiv:2509.24294.

Krithika Ramesh, Daniel Smolyak, Zihao Zhao, Nupoor Gandhi, Ritu Agarwal, Margrét V. Bjarnadóttir, and Anjalie Field. 2025. SynthTextEval: Synthetic text data generation and evaluation for high-stakes domains. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 487–499, Suzhou, China. Association for Computational Linguistics.

Sameer Sadruddin, Jennifer D'Souza, Eleni Poupaki, Alex Watkins, Hamed Babaei Giglou, Anisa Rula, Bora Karasulu, Sören Auer, Adrie Mackus, and Erwin Kessels. 2025. LLMs4SchemaDiscovery: A human-in-the-loop workflow for scientific schema mining with large language models. In *The Semantic Web – ESWC 2025*, Lecture Notes in Computer Science, pages 244–261. Springer Nature Switzerland.

Anubhav Shrimal, Aryan Jain, Soumyajit Chowdhury, and Promod Yenigalla. 2025. PARSE: LLM driven schema optimization for reliable entity extraction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 2749–2763, Suzhou, China. Association for Computational Linguistics.

Aman Singh Thakur, Kartik Choudhary, Venkat Srinik Ramayapally, Sankaran Vaidyanathan, and Dieuwke Hupkes. 2025. Judging the judges: Evaluating alignment and vulnerabilities in LLMs-as-judges. In *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM²)*, pages 404–430, Vienna, Austria and virtual meeting. Association for Computational Linguistics.

Xingbo Wang, Samantha L. Huey, Rui Sheng, Saurabh Mehta, and Fei Wang. 2025. SciDaSynth: Interactive structured data extraction from scientific literature with large language model. *Campbell Systematic Reviews*, 21(4):e70073.

Urchade Zaratiana, Gil Pasternak, Oliver Boyd, George Hurn-Maloney, and Ash Lewis. 2025. GLiNER2:

Schema-driven multi-task learning for structured information extraction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 130–140, Suzhou, China. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA. Curran Associates Inc.

A Example Schemas

Listing 1 shows an excerpt from one generated schema. Due to space constraints, we present only an illustrative portion of the full schema.

```
{
  "fields": [
    {
      "name": "case_id",
      "type_": "string",
      "description": "Unique judgment or record identifier."
    },
    {
      "name": "judgment_year",
      "type_": "integer",
      "description": "Year when the judgment was issued."
    },
    {
      "name": "outcome",
      "type_": "enum",
      "enum_values": [
        "skazanie_bez_zawieszenia",
        "skazanie_z_zawieszeniem",
        "warunkowe_umorzenie",
        "inne_nieustalone"
      ],
      "description": "Type of judgment outcome."
    },
    {
      "name": "age_years",
      "type_": "integer",
      "description": "Age in years for the primary defendant."
    },
    ...
  ]
}
```

Listing 1: Excerpt of a generated extraction schema.

B Expert-Based Evaluation Protocol

B.1 Expert Instructions

Listing 2 shows a shortened version of the annotation instructions given to the experts (the detailed instructions referenced in Section 4).

B.2 Inter-expert Coverage

Problem	Questions/expert	Inter-expert coverage
Drug offences	28.3 ± 9.6	56.4% ± 7.9pp
Medical errors	33.0 ± 13.0	52.7% ± 19.0pp
Personal rights	26.0 ± 3.0	52.3% ± 17.1pp
Overall	29.1 ± 8.8	53.8% ± 14.6pp

Table 2: Average number of questions written per expert and inter-expert coverage (Appendix B.2) per research problem.

Since each research problem was annotated by three experts independently, we can measure how much their question sets overlap. We compute *inter-expert coverage* with the same LLM-as-judge protocol used for schema coverage (Section 4), but applied between experts: for each ordered pair of experts (a, b) on the same problem, we check how many of expert a's questions are covered by expert b's question set alone. Since this is not symmetric, we compute it for both (a, b) and (b, a) for every pair, then average over all ordered pairs and problems. Table 2 reports a mean ordered-pair inter-expert coverage of 53.8%, underscoring that formulating a complete extraction schema is inherently difficult and admits no single ground truth. `Schematize` exceeds this average benchmark on several backbones (Section 4.4), meaning that its schemas cover more benchmark questions than are covered, on average, by another expert's question set; this does not imply superiority to every individual expert.

```
Annotator Instructions Summary
Role and Task:
The annotator creates a set of questions that a data extraction
       schema must answer based on Polish court judgments. These
       questions act as a benchmark to verify if the automated
       schema captures all key information defined by the expert.
Research Problem Definition:
The problem is defined via a three-part dialogue:
    1. user_input: The researcher describes the research goal.
    2. problem_help: The bot asks clarifying questions.
    3. user_feedback: The researcher confirms, rejects, or adds
          details.
Questions must stem from the entire dialogue, covering only the
       confirmed scope and ignoring rejected topics.
Criteria for Valid Questions:
Questions must determine what the system can extract for
       quantitative analysis.
    1. Extractability: Questions must be answerable directly from a
          single judgment's text, without external knowledge or
         legal doctrine.
    2. Unambiguity: Questions must be clear in intent. Subjective
          evaluations are permitted if grounded in the text.
    3. Simplicity: One question equals one fact. Multi-threaded
          questions must be split.
    4. Coverage: The questions must comprehensively cover all
          aspects confirmed in the dialogue.
Answer Formats:
Acceptable formats include Yes/No, Number, Scale, Text, and
       Category. If categorical options are not exhaustive, use a
       series of Yes/No questions instead.
```

Listing 2: Shortened version of the expert annotation instructions.

C Detailed results

While Table 1 averages coverage across the three research problems, Table 3 reveals substantial variation across both problems and backbones. Averaged across backbones, ScheMatiQ + PDH performs best on drug offences (70.0%), whereas `schematize` performs best on medical errors (53.3%) and personal rights (63.1%). Across the 21 backbone-problem combinations, `schematize` ranks first in 13 and first or second in 17. Its performance is particularly consistent on personal rights, where it leads for six of the seven backbones, and on medical errors, where it ranks among the top two for every backbone. Results on drug offences are more mixed: although `schematize` wins for three backbones, ScheMatiQ + PDH achieves the highest cross-backbone average.

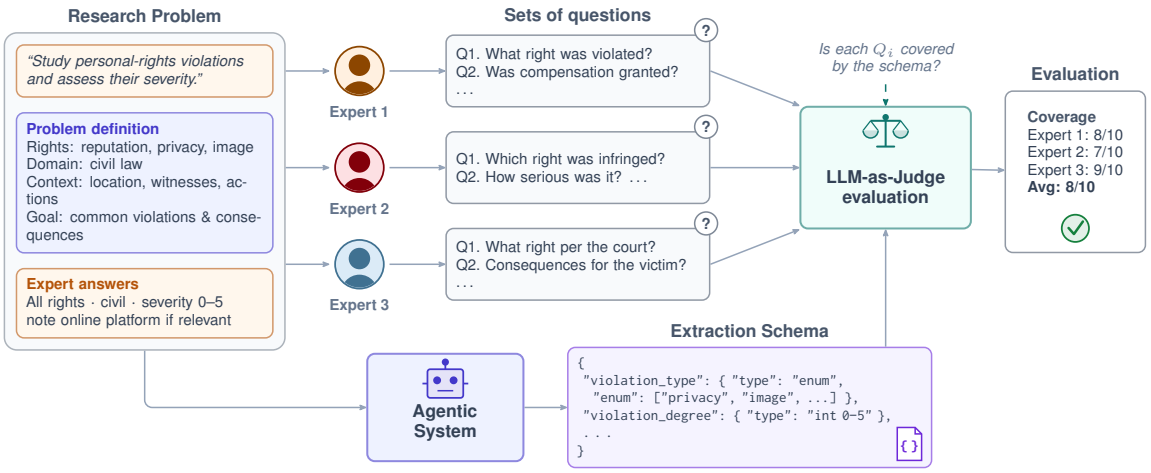


Figure 5: Expert-based evaluation pipeline. Given a shared research problem, each domain expert writes an exhaustive question set the extracted data should answer, while the agentic system independently produces an extraction schema. An LLM-as-judge checks whether each expert question is covered by the schema, yielding per-expert and average coverage scores.

Backbone	Drug offences					Medical errors					Personal rights				
	vanilla LLM	vanilla LLM + PDH	Sche MatiQ	Sche MatiQ + PDH	schematize	vanilla LLM	vanilla LLM + PDH	Sche MatiQ	Sche MatiQ + PDH	schematize	vanilla LLM	vanilla LLM + PDH	Sche MatiQ	Sche MatiQ + PDH	schematize
gpt-5.4	49.0%	87.0%	59.3%	84.3%	**94.0%**	25.9%	53.4%	28.1%	57.2%	**71.5%**	33.9%	52.5%	43.9%	51.2%	**72.8%**
gpt-5.4-mini	38.0%	72.7%	15.9%	66.3%	**83.2%**	23.9%	47.9%	31.7%	54.1%	**70.6%**	33.3%	46.0%	24.1%	52.4%	**70.6%**
gpt-5.4-nano	41.2%	**71.5%**	9.2%	63.6%	1.9%	20.0%	**52.3%**	26.1%	22.8%	30.7%	5.5%	55.6%	36.6%	41.0%	**71.4%**
claude-sonnet-4.6	49.5%	74.9%	82.6%	84.1%	**89.9%**	29.7%	50.1%	27.3%	**57.2%**	53.5%	32.1%	56.4%	38.8%	59.9%	**66.7%**
qwen3.6-35b-a3b	27.9%	54.5%	47.3%	**84.0%**	83.9%	20.2%	50.1%	22.6%	**54.0%**	51.5%	37.5%	50.2%	51.4%	63.5%	**64.0%**
gemma-4-e4b-it	0.0%	**55.0%**	53.0%	52.0%	50.1%	3.4%	26.9%	16.6%	43.2%	**57.3%**	7.9%	32.8%	33.9%	44.4%	**52.9%**
llama-4-scout-17b	25.0%	50.1%	54.2%	**55.8%**	53.7%	9.2%	32.7%	16.8%	35.6%	**38.3%**	**48.8%**	43.2%	27.3%	44.4%	43.2%

Table 3: Per-problem expert-question coverage (%) across backbones and configurations; columns match Table 1. Bold marks the best, underline the second-best, configuration per backbone and problem.

Ablation	Drug offences	Medical errors	Personal rights	Overall
Full (schematize)	83.9%	51.5%	64.0%	66.5%±16.3pp
w/o PDH	-31.5pp	+5.5pp	-16.9pp	-14.3pp±18.7pp
w/o CBR	+0.1pp	+0.2pp	+4.4pp	+1.5pp± 2.5pp
w/o DGR	+0.7pp	+3.9pp	-7.4pp	-1.0pp± 5.8pp
w/o PDH+CBR	-36.2pp	-7.4pp	-4.3pp	-16.0pp±17.5pp
w/o PDH+DGR	-29.4pp	-13.7pp	-18.8pp	-20.6pp± 8.0pp
w/o CBR+DGR	-40.0pp	-6.2pp	-9.6pp	-18.6pp±18.6pp
w/o PDH+CBR +DGR	-83.9pp	-33.1pp	-43.3pp	-53.4pp±26.9pp

Table 4: Expert-question coverage after selectively ablating components of the schematize pipeline. Entries show the change in coverage (percentage points, pp) relative to the full system, averaged over the research problems.

Backbone	vanilla LLM	ScheMatiQ	schematize
gpt-5.4	$0.024 ± 0.008	$0.176 ± 0.076	$3.785 ± 0.312
gpt-5.4-mini	$0.005 ± 0.000	$0.044 ± 0.020	$0.828 ± 0.059
gpt-5.4-nano	$0.002 ± 0.001	$0.014 ± 0.007	$0.234 ± 0.015
claude-sonnet-4.6	$0.027 ± 0.006	$0.243 ± 0.086	$3.057 ± 0.510
qwen3.6-35b-a3b	$0.005 ± 0.001	$0.033 ± 0.007	$0.183 ± 0.010
gemma-4-e4b-it	$0.000 ± 0.000	$0.016 ± 0.001	$0.041 ± 0.001
llama-4-scout-17b	$0.000 ± 0.000	$0.007 ± 0.001	$0.023 ± 0.002

Table 5: The LLM inference cost in USD per research problem ($mean \pm std$), per backbone, for the *vanilla LLM*, *ScheMatiQ*, and schematize configurations.

C.1 Ablation

D Costs

Table 5 reports the average inference cost per research problem for each configuration. Schematize is consistently the most expensive configuration, reflecting its multi-agent, multi-round design, while ScheMatiQ costs roughly an order of magnitude less than schematize and vanilla LLM is cheapest by a further order of magnitude. This gap is most pronounced for large closed-weight backbones (e.g., gpt-5.4, claude-sonnet-4.6) and narrows substantially for smaller open-weight models (e.g., gemma-4-e4b-it, llama-4-scout-17b), where schematize's absolute cost drops to a few cents per problem. Combined with the coverage results in Table 1, this suggests that schematize's largest coverage gains come at a real cost premium, but that premium can be kept small by pairing schematize with a smaller backbone rather than the largest available model.